

Apache Spark Core

源码深度分析

核心流程图 & 关键类设计说明
基于 Spark 3.3.1 源码

版本: v3.3.1 | 模块: spark-core

目录

第一章 Spark Core 整体架构概览

1.1 模块组织结构

1.2 整体架构图

1.3 核心组件关系

第二章 核心流程图

2.1 Job提交与执行流程

2.2 Stage划分与依赖分析

2.3 任务调度与执行流程

2.4 Shuffle机制

2.5 统一内存管理

2.6 BlockManager存储

2.7 故障处理与重试

第三章 核心类设计说明

3.1 SparkContext

3.2 DAGScheduler

3.3 TaskSchedulerImpl

3.4 Executor

3.5 RDD

3.6 BlockManager

3.7 UnifiedMemoryManager

3.8 SortShuffleManager

3.9 RPC框架

3.10 SparkEnv

第四章 关键设计模式总结

第一章 Spark Core 整体架构概览

1.1 模块组织结构

Spark Core 是整个 Apache Spark 的基石, 位于 core/ 目录下, 提供分布式任务调度、内存管理、容错恢复、存储管理和 I/O 等核心能力。所有上层模块 (SQL、Streaming、MLlib、GraphX) 都构建在 Core 之上。

模块目录	核心职责	关键类
scheduler/	任务调度(DAG/Task)	DAGScheduler, TaskSchedulerImpl, TaskSetManager
executor/	任务执行	Executor, TaskRunner
rdd/	RDD抽象与实现	RDD, MapPartitionsRDD, ShuffledRDD
storage/	块存储管理	BlockManager, MemoryStore, DiskStore
shuffle/	Shuffle读写	SortShuffleManager, ShuffleWriter/Reader
memory/	内存管理	UnifiedMemoryManager, MemoryPool
rpc/	RPC通信	RpcEnv, RpcEndpoint, RpcEndpointRef
deploy/	部署(Standalone)	Master, Worker, SparkSubmit
broadcast/	广播变量	TorrentBroadcast, BroadcastManager
serializer/	序列化	KryoSerializer, JavaSerializer
util/	工具类	Utils, ClosureCleaner, EventLoop

1.2 整体架构图

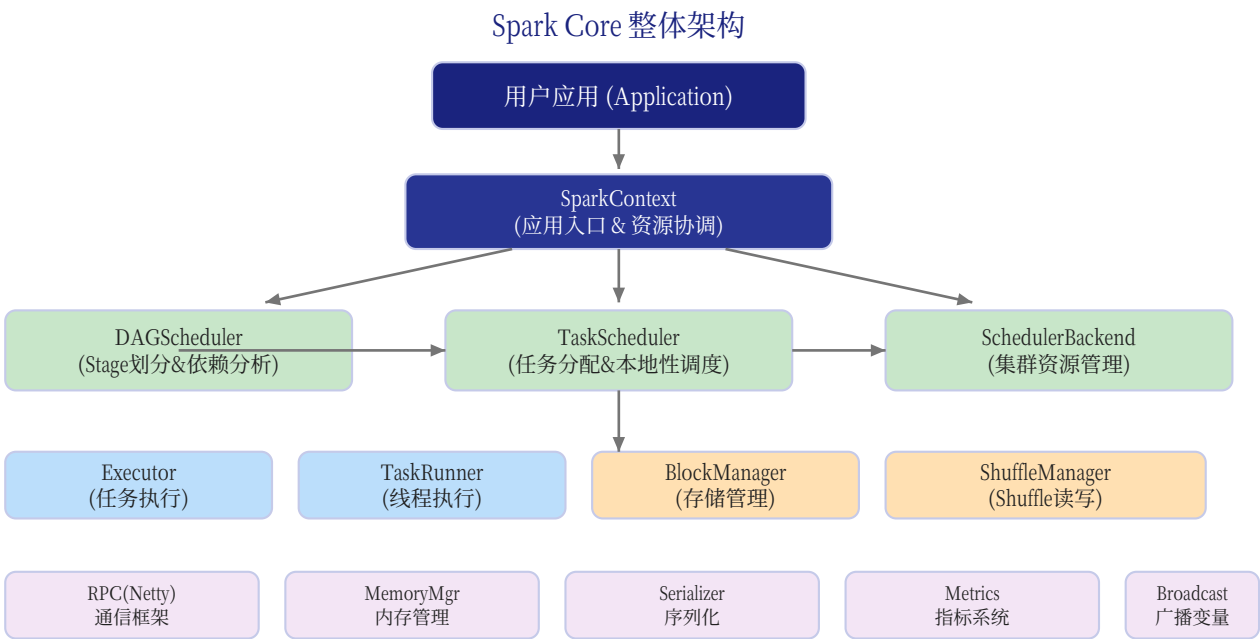


图 1-1: Spark Core 分层架构

1.3 核心组件关系

Spark Core 的设计遵循分层架构, 从上到下分为四个层次:

- 应用层: SparkContext 作为统一入口, 管理应用生命周期
- 调度层: DAGScheduler(Stage级) + TaskScheduler(Task级)
- 执行层: Executor执行任务, BlockManager管理存储, ShuffleManager管理重分布
- 基础设施层: RPC通信、内存管理、序列化、指标系统

第二章 核心流程图

2.1 Job 提交与执行流程

当用户调用 RDD 的 Action 操作时, 会触发 Job 提交。流程从 SparkContext.runJob() 开始, 经 DAGScheduler 的 Stage 划分和 TaskScheduler 的任务分配, 最终在 Executor 上执行。

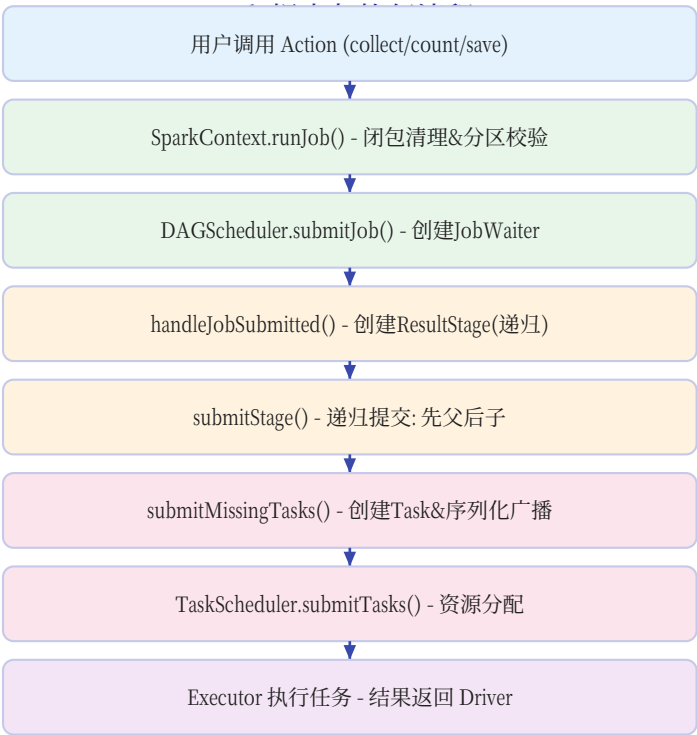
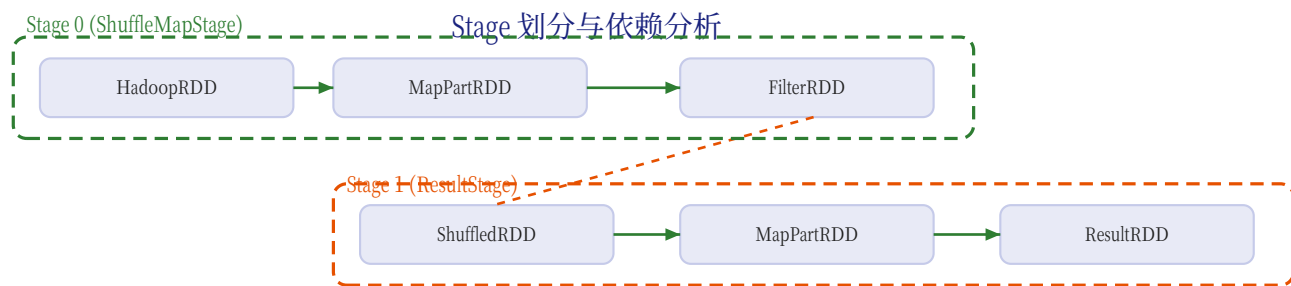


图 2-1: Job 从提交到执行的完整流程

2.2 Stage 划分与依赖分析

DAGScheduler 按 Shuffle 边界切分 Stage。窄依赖在同一 Stage 内流水线执行, 宽依赖触发 Stage 划分。

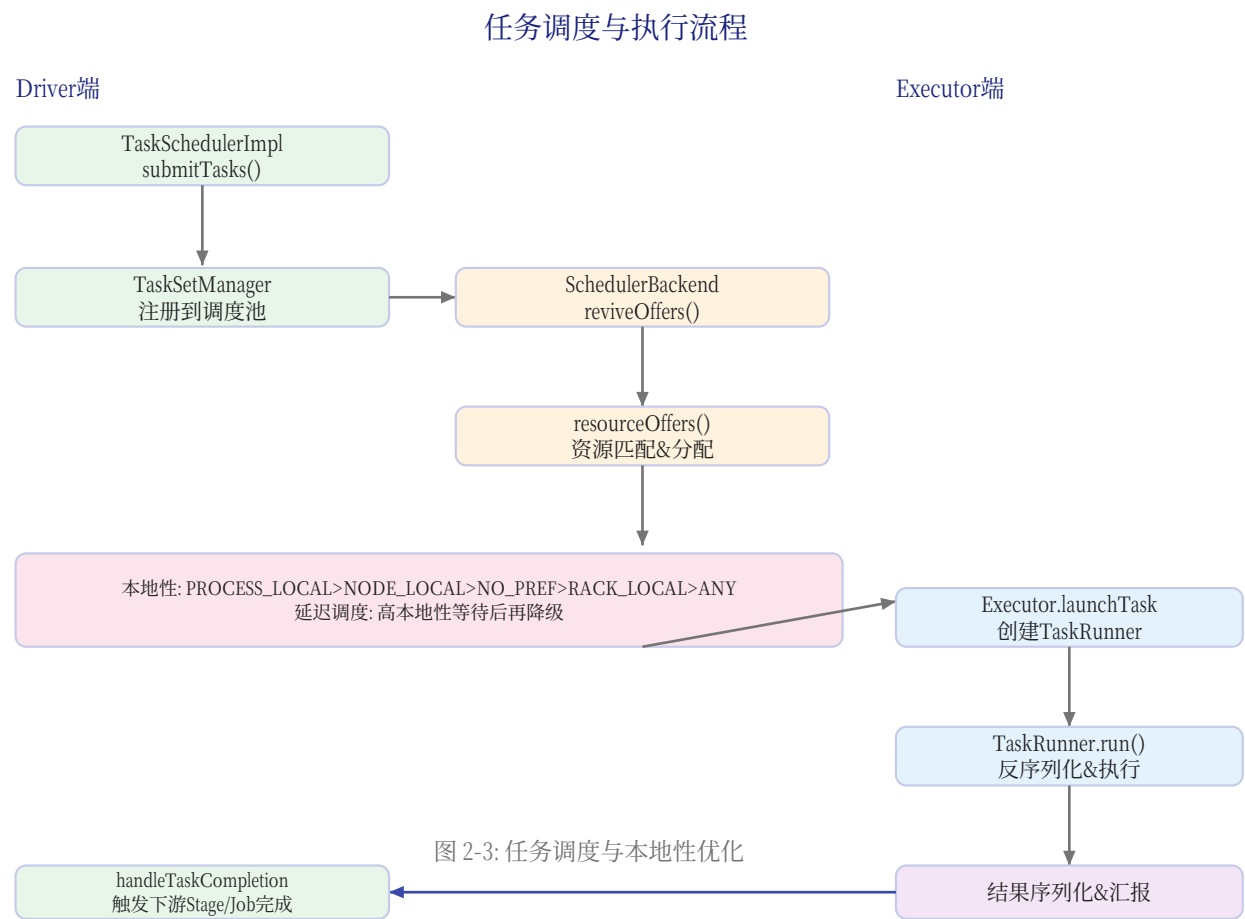


窄依赖(NarrowDep): 同Stage流水线执行 宽依赖(ShuffleDep): Stage边界,需Shuffle

图 2-2: Stage 划分与窄/宽依赖

2.3 任务调度与执行流程

任务调度涉及 Driver 端的 TaskSchedulerImpl/TaskSetManager 和 Executor 端的 Executor/TaskRunner 协作。延迟调度是性能优化的关键。



2.4 Shuffle 机制与三种写入路径

SortShuffleManager 根据条件选择三种写入路径。所有路径输出统一格式, 读取路径统一使用 BlockStoreShuffleReader。

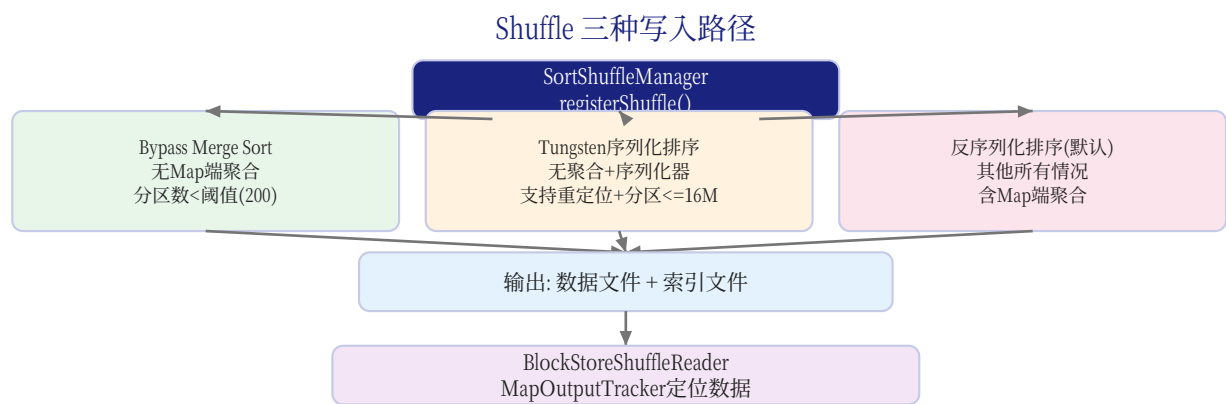


图 2-4: Shuffle 三种写入路径

2.5 统一内存管理模型

UnifiedMemoryManager 的核心特征是执行与存储内存之间的"软边界", 双方可互借但遵循不对称规则。



可用内存 = (系统内存-300MB) x spark.memory.fraction(0.6)
存储区域 = 可用内存 x spark.memory.storageFraction(0.5)
执行区域 = 可用内存 - 存储区域

不对称借用规则:

1. 执行向存储借: 可强制驱逐存储中的缓存块(缓存可重算)
2. 存储向执行借: 只能借执行的空闲内存(不能驱逐执行内存)

图 2-5: 统一内存管理模型

2.6 BlockManager 存储与检索

BlockManager 管理 RDD 缓存、广播变量和 Shuffle 数据。存储"先内存后磁盘", 检索"先本地后远程"。

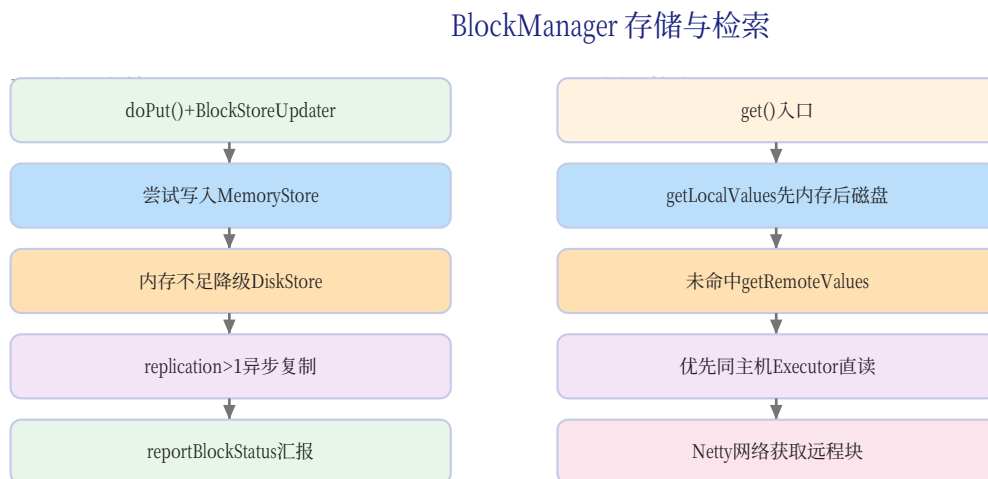


图 2-6: BlockManager 存储与检索流程

2.7 故障处理与重试机制

不同失败类型有不同处理策略: FetchFailed 重新提交上游 Stage, ExceptionFailure 重试 Task, Barrier 失败重试整个 Stage。

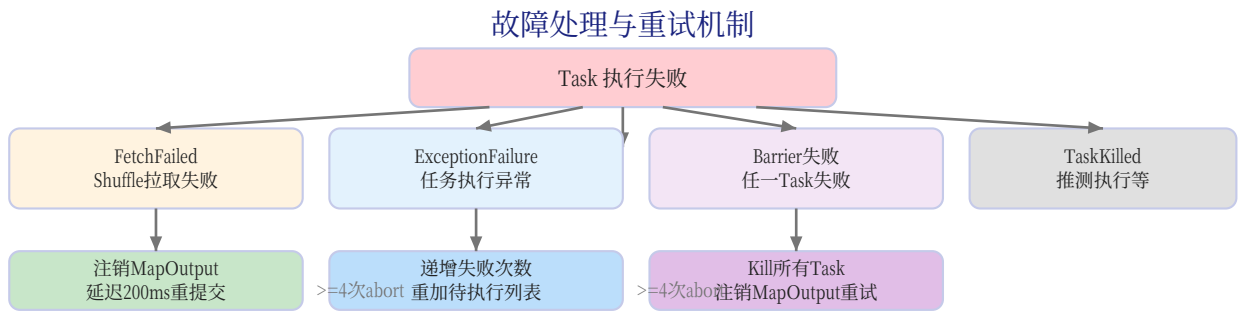


图 2-7: 故障分类与重试策略

第三章 核心类设计说明

3.1 SparkContext -- 应用入口

SparkContext 是 Spark 应用程序的主入口点(约3,250行), 负责初始化所有运行时组件并管理应用完整生命周期。

核心职责:

- 初始化约30个核心组件(SparkEnv、DAGScheduler、TaskScheduler等)
- 提供所有Action操作的入口(runJob/submitJob)
- 管理应用生命周期: 创建->运行->停止
- 通过工厂方法适配不同集群管理器(Local/Standalone/YARN/K8s)

初始化四阶段:

阶段	关键操作	核心组件
1.配置与基础	校验SparkConf,解析资源	_conf, _resources, _jars
2.事件与环境	创建运行时环境	ListenerBus, SparkEnv, SparkUI
3.调度器初始化	创建调度器和心跳	DAGScheduler, TaskScheduler, HeartbeatReceiver
4.启动与后置	启动调度器,初始化BlockMgr	BlockManager, MetricsSystem, EventLogger

runJob() 是所有 Action 的统一入口(7个重载), 核心流程: 检查未停止 -> 闭包清理 -> dagScheduler.runJob() -> RDD checkpoint. stop() 以严格逆序停止所有组件, AtomicBoolean 保证幂等。

3.2 DAGScheduler -- DAG 调度器

DAGScheduler(约3,625行) 是高层调度器, 将 RDD DAG 按 Shuffle 边界切分为 Stage, 根据依赖关系决定提交顺序。

数据结构	类型	用途
waitingStages	HashSet[Stage]	父Stage未完成,等待中
runningStages	HashSet[Stage]	当前正在运行
failedStages	HashSet[Stage]	需重新提交
stageIdToStage	HashMap	StageID到Stage映射
shuffleIdToMapStage	HashMap	ShuffleID到ShuffleMapStage映射

核心 workflow: "自顶向下构建 Stage DAG -> 自底向上提交执行 -> 结果自底向上传播触发下游 -> 失败时回退重试"

3.3 TaskSchedulerImpl -- 任务调度器

TaskSchedulerImpl(约1,308行) 是低层调度器, 将 Task 分配到具体 Executor。核心是 resourceOffers() 方法:

- â─ 过滤排除节点(HealthTracker)
- â─ 随机打乱Offers确保均衡
- â─ 按策略排序TaskSet(FIFO/FAIR)
- â─ 逐级本地性分配: PROCESS_LOCAL->NODE_LOCAL->NO_PREF->RACK_LOCAL->ANY
- â─ Barrier任务必须同一轮全部分配成功

延迟调度: 在高本地性级别等待一段时间(spark.locality.wait.*), 等不到才降级, "牺牲公平性换数据本地性"。

3.4 Executor -- 任务执行器

Executor(约1,775行) 运行在 Worker 节点, 通过线程池并发执行 Task。TaskRunner.run() 流程:

- â─ 环境准备: 创建TaskMemoryManager, 设置类加载器, 汇报RUNNING
- â─ 依赖加载: 下载JAR/文件/归档(增量更新)
- â─ Task反序列化: 从Driver广播数据恢复Task对象
- â─ 执行计算: task.run()触发RDD计算链
- â─ 结果处理: 按大小三级策略(直接返回/存BlockManager/丢弃)

结果大小	策略	说明
>maxResultSize(1GB)	丢弃	仅返回IndirectTaskResult
>maxDirectResultSize	存BlockManager	返回间接引用
较小	直接RPC发送	序列化后Netty返回Driver

3.5 RDD -- 弹性分布式数据集

RDD(约2,710行) 是 Spark 最核心的数据抽象, 不可变、可分区、可并行计算。五大核心属性:

属性	方法	说明
分区列表	getPartitions	数据切分为多分区并行处理
计算函数	compute(split,ctx)	定义如何计算每个分区
依赖列表	getDependencies	对父RDD的依赖(容错血缘)
分区器	partitioner	K-V数据分区方式(可选)
首选位置	getPreferredLocations	数据本地化提示(可选)

依赖体系:

依赖类型	子类	特征	示例操作
窄依赖	OneToOneDependency	子分区只依赖一个父分区	map, filter, flatMap
窄依赖	RangeDependency	连续分区范围映射	union
宽依赖	ShuffleDependency	父分区被多个子分区使用	reduceByKey, join

MapPartitionsRDD: 最常用的RDD(map/filter/flatMap底层), compute()对父RDD迭代器应用转换, 窄依赖流水线。ShuffledRDD: Shuffle产生的RDD(reduceByKey/join), compute()通过ShuffleReader从Map输出读取。

3.6 BlockManager -- 存储管理器

BlockManager(约2,400行) 运行在每个节点, 管理RDD缓存、广播变量和Shuffle数据, 协调MemoryStore和DiskStore两级存储。

- ⚡ 先内存后磁盘: Put时优先MemoryStore, 不足降级DiskStore
- ⚡ 先本地后远程: Get时先本地(内存->磁盘), 未命中再远程
- ⚡ 退役迁移: 节点退役时BlockManagerDecommissioner迁移块
- ⚡ 读写锁: BlockInfoManager提供块级读写锁机制

3.7 UnifiedMemoryManager -- 内存管理器

UnifiedMemoryManager(约493行) 的核心创新是"软边界": 执行和存储可互借, 但不对称 -- 执行可强制驱逐存储缓存(安全, 可重算), 存储只能借执行空闲内存(不能驱逐, 因需中断任务)。

场景	行为	原因
执行向存储借	可强制驱逐缓存块	缓存可重算恢复, 安全
存储向执行借	只能借空闲内存	驱逐执行内存需中断任务, 复杂

3.8 SortShuffleManager -- Shuffle 管理器

SortShuffleManager(约273行) 唯一内置实现, 三种路径:

路径	条件	Writer
Bypass Merge Sort	无Map聚合+分区<200	BypassMergeSortShuffleWriter
Tungsten序列化排序	无聚合+序列化器支持重定位	UnsafeShuffleWriter
反序列化排序(默认)	其他所有(含Map聚合)	SortShuffleWriter

Tungsten优化: 二进制排序减少GC; 8字节指针提升缓存命中; 溢写合并无需反序列化; NIO transferTo高效复制。

3.9 RpcEnv & RpcEndpoint -- RPC 通信框架

基于 Netty 的 Endpoint/EndpointRef/RpcEnv 模式, 所有组件间通信都通过该框架完成。

组件	角色	关键方法
RpcEnv	消息总线/路由器	setupEndpoint, setupEndpointRef
RpcEndpoint	消息处理器	receive, receiveAndReply, onStart/Stop
RpcEndpointRef	远程引用/代理	send(单向), ask(请求-响应)

3.10 SparkEnv -- 运行时环境

SparkEnv(约609行) 持有一个Spark实例所需全部运行时组件, Driver和Executor各自创建独立实例:

序号	组件	说明
1	SecurityManager	安全认证与加密
2	RpcEnv(Netty)	RPC通信环境
3	Serializer+SerializerManager	数据序列化
4	BroadcastManager	广播变量管理
5	MapOutputTracker	Map输出位置跟踪(Master/Worker)
6	BlockManagerMaster+BlockManager	分布式块存储管理
7	MetricsSystem	指标收集
8	OutputCommitCoordinator	输出提交协调

第四章 关键设计模式与思想总结

1. 事件驱动架构

DAGScheduler 通过 EventLoop 处理所有调度事件, 单线程串行保证线程安全; LiveListenerBus 异步分发事件, 解耦核心逻辑与监控/UI。

2. 分层调度设计

DAGScheduler(逻辑调度, Stage级) + TaskScheduler(物理调度, Task级) 的两层分离, 使得调度策略可以独立演进。

3. 血缘容错机制

RDD 通过 Dependency 记录完整血缘, 分区丢失时从最近的完整数据重新计算, 无需数据复制, 存储开销极低。

4. 软边界内存管理

执行和存储内存共享同一空间, 通过不对称借用规则动态调整, 避免静态分配导致的资源浪费。

5. 延迟调度优化

在数据本地性和调度公平性之间取得平衡, 先在数据所在位置等待, 等不到再扩大搜索范围。

6. 可插拔架构

ShuffleManager、Serializer、SchedulerBackend 等核心组件都可通过配置替换, 支持不同场景的定制。

7. 推测执行容错

TaskSetManager 定期检查慢任务, 对运行时间超过中位数 1.5 倍的任务启动推测副本, 取先完成者结果。

8. 多级结果返回

Executor 根据结果大小选择直接返回/存BlockManager/丢弃三级策略, 避免大结果打爆 Driver 内存。

9. 异步复制存储

BlockManager 支持异步跨节点块复制和退役迁移, 提高数据可用性和集群弹性。

10. 统一通信框架

基于 Netty 的 RPC 框架统一了所有组件间通信, 支持单向消息和请求-响应两种模式, 简化了分布式系统开发。